Consider it done.

A service ticket with a durable permission boundary.

This is an offline, single-user home-service agent. Python 3.11+ standard library serves a browser UI and owns a SQLite state machine. No model, provider network, credential, payment or external service is required. `Agent.snapshot()` and `Agent.act()` implement the documented API.

Boundaries

`interpreter.py` reads only the user's text. It extracts supported service, location, relative day, time and INR budget facts. It never receives provider reviews. The state machine asks for missing facts and uses structured ticket edits for precise correction. `tools.py` implements the assignment's eight fixed provider tools. `store.py` persists sessions, preferences, approvals, holds, operation journals and authoritative provider results. Provider identity is separate from session identity: the stable local user sees bookings across errands; approvals still bind the session that displayed the ticket.

Search produces four mock providers. The agent applies budget, time-window, earliest-time and technician constraints. Selection fetches provider details and creates a ten-minute hold. The displayed amount comes from `hold_slot.quoted_price`, including the configured price difference, and must fit the budget. Provider review text deliberately contains an injection attempt; it remains inert data.

Approval and recovery

A proposal fingerprint hashes session, action, provider, service, exact slot, exact price, hold, expiry and revision. Execution recomputes that fingerprint and checks the durable hold tuple before creating an approval record. Ticket edits erase the old proposal. Approval and operation key commit before provider mutation. Provider booking and idempotency receipt commit atomically in a separate transaction. This transaction split makes the interruption window real and recoverable.

Repeated confirmation looks up the existing journal key. A timeout triggers `get_booking_status` before success or replacement. Recovery after reopening SQLite reuses the persisted approved operation; an authoritative missing result permits a bounded replay with the same key. Unknown status blocks replacement, including attempts to start another errand. Cancellation's first 503 gets one retry. Reschedule conflict preserves the original booking. The receipt includes the verified booking ID.

A unique provider/slot occupancy table is checked inside an immediate write transaction. Independent holds and operation keys cannot double-book one provider slot. Cancellation releases occupancy; a reschedule checks its new slot before releasing the original. Search and hold also omit occupied availability.

Limits and memory

One offline interpreter turn, eight tool calls and three seconds wall time bound a turn. Limits are configurable on the engine for deterministic tests. Each tool runs in a daemon worker with a deadline and its own SQLite connection. The user-facing call returns at the deadline even if a tool is still settling. A durable pending operation and an in-flight gate block replacement; late mutation is reconciled through authoritative status before claiming completion. A worker cannot forcibly undo an already-started effect. All handled failures produce an assistant message; unexpected exception details are not sent to the browser.

Area, earliest time and preferred technician have stated/inferred provenance. Inferred memory requires explicit confirmation before new-session reuse. Forget deletes the persisted preference and removes inherited values and dependent proposals from active sessions. Directly entered errand facts remain explicit request facts.

Observability and limitations

Traces record transitions, sanitized tool arguments/results, attempted tool exceptions, latency, approval and one interpreter event per chat. Token and cost fields are null with an explicit no-external-model explanation. All run events persist in SQLite's traces table; snapshots retain the latest 250 events. Chat history remains user content, while traces redact recognizable API keys and secret fields.

The interpreter is intentionally deterministic: supported services are plumbing, electrical, AC and cleaning; dates include today, tomorrow, this/next week, weekend, weekday names and structured ISO dates. Search chooses a representative available day within the supported range, rather than enumerating every day. An unsupported conversational request may need a structured correction. Bookings, preferences and session data are local to the chosen database. This is a local demo, not multi-user authentication or a production scheduling system.

Verification: `python run_evals.py` executes offline behavioral tests and generates the actual result table and raw timeout-after-success trace under `output/`.

The recorded timeout trace reads: `approval > executing; book > timeout; executing > reconciling; get_booking_status > booked + booking_id; reconciling > completed`. The timeout is transport uncertainty; only the authoritative booking result produces a receipt. The operation key is identical in book and status calls.

At 100x usage, separate provider workers from HTTP processes, retain the durable outbox/idempotency boundary, move SQLite writes to a transactional service database and index per-user operations and run traces. Add authentication, per-user authorization and retention controls before serving multiple people. Preserve exact approval tuples and authoritative reconciliation rather than increasing retries.